
Requirements

This document: [Requirements-Charlie-2.pdf](#)¹.

1. No X86 (other than Debugging) [Agreed]
2. No interrupt service (premature optimization). [The jamming signal is a later optimization we can add. We will start with 16 byte packets]
3. Flat-out ship slices and frames around at top speed.
4. Basic State Machine using link-based-reversible-black-tokens supporting applications using node-based-irreversibly-colored-tokens. [Agreed – specified in this document, and in [State-Machine-Specification](#).
5. Then add features [9 machines in a tile demo, the liveness tensor, link-based-reversible-black-tokens and node-based-irreversibly-colored-tokens will only be proven at scale]
6. Liveness between A & B on a link requires 3 Pertri-net places on A and 3 on B to be reversible. [Static Globals in HLS]
7. Atomic transfer between A & B on a link requires 6 places on A and 6 on B to be reversible. [Atomic transfer Is equivalent to the 4 state protocol. Link-based-reversible-black-tokens will need to be specified as primitives available to support applications using node-based-irreversibly-colored-tokens.]
8. Neither of those, nor any extension of Spekkens in $N = 1 +$ combination will require more than $N * 3$ places on A or $N * 3$ places on B. [Extensions will need their own metadata and data]
9. in particular, changes to the liveness tensor will be priority deferred until combined Spekkens extension operations are complete or reversed. [Yes. This is essential]

Black-Tokens

-

Plan

Our ultimate target for now is the [Kria LR260 Robotics Starter Kit](#) (\$349 for a card – starter kit). 10-15-20 cards, order them in an array, look at ...

[Manual/DataSheet](#).

[Developer Information](#).

Use [Meta Mako \(Arista\) L1 switch](#).

Rewire everything on the fly can jam things.

Our initial target for now is the Xilinx ALVEO U50 card and that means Vitis and Vivado on the hardware side and XRT on the host computer side (either X86 or ARM64) as the implementation development environment. That allows us once we have code to port it immediately to the Kria, which is supported by the same development environment.

Configuration for Demo: 4-way mesh, using the Kria LR260 Boards connected on a tabletop. Two nodes are required for testing a single link. Three nodes are needed for a triangle Demo. 9 nodes are needed for a basic 3x3 tile. 16 nodes are needed for a 4x4 tile (with tile failover capability), and $5 \times 5 = 25$ nodes are needed for a scalable test within one rack.

¹If you put all these specification documents in one folder, the links will work in Latex and other pdf reader programs to open those documents when you click on them).

Principles of Operation

The first step in the protocol is to establish roles. One side of the link becomes Alice, the other side becomes Bob. This is described in the [slice-engine.pdf](#) under initialization.

The slice engine then continuously ‘exchanges’ (as fast as possible) in a shared (circulated) data structure across a link between Alice and Bob. Frames are composed of 8 byte slices. There are a maximum of 8 slices (64 bytes) in a frame. SerDes Limitations will require that two slices are sent at a time.

Register Transfer Architecture

The architecture avoids buffers (but not FIFO buffers, which are needed for forwarding and deferred or out-of-band execution). The stringent Atomicity requirements of the entangled state model do not play well with anything that is ‘variable length’. The basic communication architecture is a continuous / bidirectional transferring of registers from Alice to Bob, and Bob to Alice, in a continuous circular fashion.

Frames, Slices and Link-Based-Reversible-Black-Tokens

Two Atomic Frames

There are two completely independent (concurrent) Frames. One owned by Alice (which only Alice can initiate and tear-down), and one owned by Bob (which only Bob can initiate and tear down).

Inside each frame, on the first slice there are two completely independent (epistemic) ‘views’ of Petri black tokens, one view is from Alice’s side, and one view is from Bob’s side. See the [Dual-SAW-Spekkens.pdf](#)

Inside each independent frame there are two entangled Petri black tokens: one owned by Alice (4-bits), and One Owned by Bob (4-bits). Individually, they are used by their owner to ‘control’ the progress of the execution(s) they own. For example, only Alice can transition her black token from the liveness function to the atomic transfer state machine function. All that Bob can do with Alice’s black token is to participate with his computational resources in the state machine specified (1, 2, and 4) roundtrips to complete the protocol. Also, only Bob can transition his black token from the liveness function to the atomic transfer state machine function. All that Alice can do with Bob’s black token is to participate with her computational resources in the state machine specified (1, 2, and 4) roundtrips to complete the bounded protocol sequence.

This guarantees that neither side can stall the protocol indefinitely (the maximum number of round trips is 4 – we need only 2 for a simple atomic transfer)²

Link Liveness Black Tokens

There are 2 Link Liveness fields³, 1 on Alices’s side (1 of the 4 bytes Alice receives and can modify, and 1 of the 4 bytes Bob has sent which Alice may not modify). Giving 2 ‘across link’ views of the link state.

There are a total of 16 bits of **Link Liveness** in the first slice (8 bytes) of the (potentially 64-byte) Frame. We use two black tokens. One owned by Alice, and one Owned by Bob. There are 8 bits total in each **Link Liveness** field, 4 bits for Alice’s black token, and 4 bits for Bob’s black token.

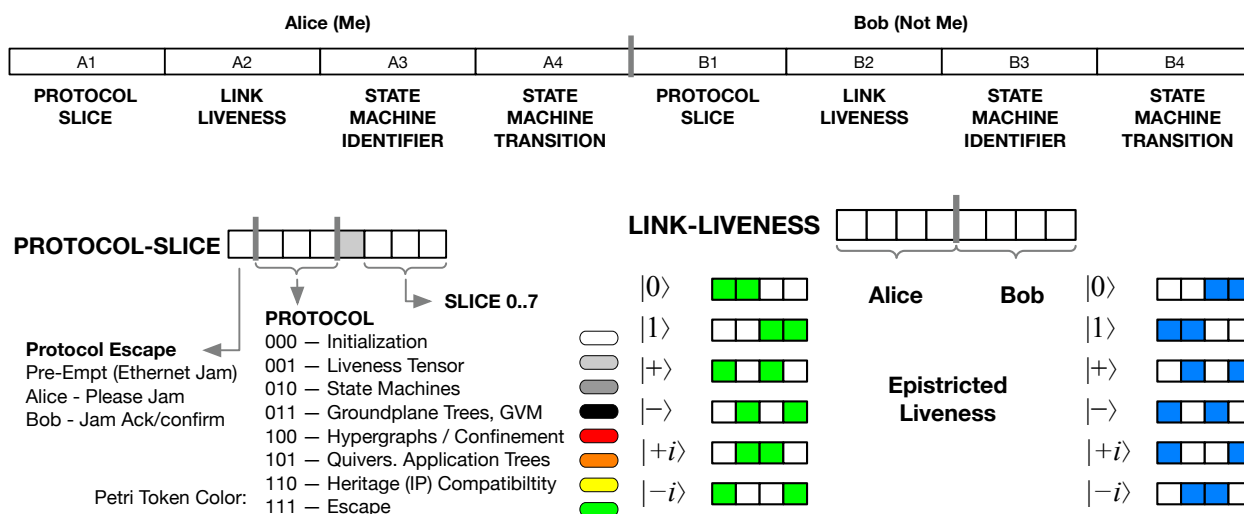
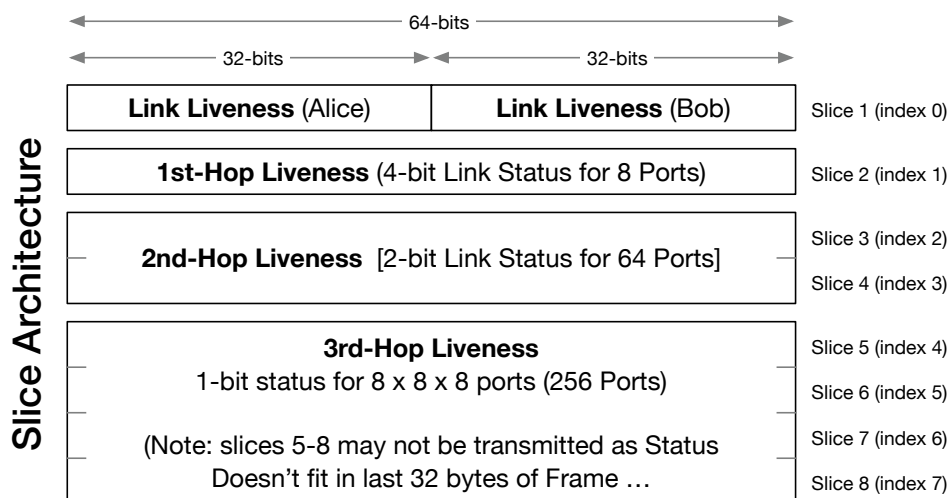
This specification defines a family of local mesh architectures, designed to (1) take advantage of short cables ($\leq 0.5m$) between cells in a single rack, and (2) to pave the way to a local addressing scheme that can provide a more secure foundation than current L2 Any to Any (A2A) protocols. We retain a great deal of the Ethernet philosophy; e.g. the basic 64-byte frame, and try to incorporate what the industry has learned since it was first introduced in 1980 and standardized by the IEEE in 1983.

²The Protocol has space for a maximum of 8 roundtrips for more sophisticated exchanges in a future version of the protocol. Even then, the each side cannot hang on to the black token without visibly violating the protocol and alerting their neighbors of a fault condition – or misbehavior of the software.

³A conventional Stop and Wait (SAW) protocol uses 1 black token, and provides an ‘alternating causality’ mechanism on a single link. The SAW protocol is frequently extended to be a credit based flow control mechanism, with 0, 1 or N places in the buffer.

Special requirements

1. Single/Dual Black Tokens. The testing requires that we create both situations in testing (1) where there is just one unary black token (because either Alice or Bob started first), and (2) where there are two initial black tokens, starting simultaneously that correctly give us binary black atomic communication reliably. And, this requires special synchronization code for testing to work. [As in Pauli's electrons (Fermions) these black tokens are indistinguishable and only placing them in exclusive states (Alice-owned or Bob-owned) allows us to correctly identify them, even after a link failure and Alice and Bob ends of the link switching their link identities during symmetry breaking to Bob and Alice.]
2. In the case of a completely brand new system being introduced to the mesh, because the previous system bricked leaving no state, we have to replace him, which will involve the software coming up and all of his state being renewed from all the data sources and replicated state that is out there. Before we do this we must vet the new node to avoid various exploits, hacks, etc. This is a job for Verilock to tell us how to do this in the absence of fingerprints, or in the presence of a special fingerprint from them with a node initialization from Verilock.



Change "Petri Token Color" (above) to "Petri Black Token" in image, and we are not doing jamming until later.

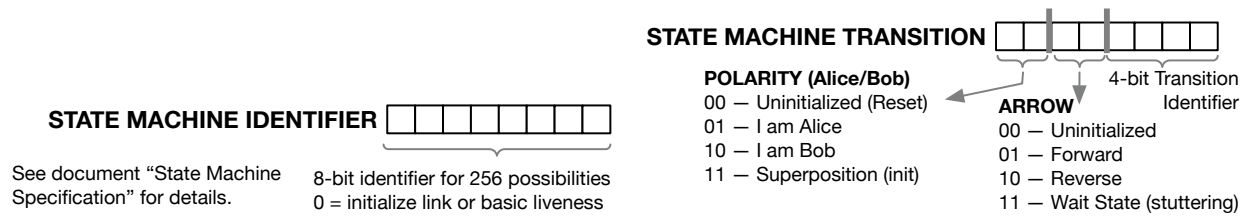


Figure 1: Slice Engine. (See [Dual SAW Spekkens](#) for details)

The slice engine manages frames, made up of 8 Byte *slices*, numbered 1 to 16 (indexed 0..15). 8 slices represent one 64-byte frame, 16 slices represent two atomically-consecutive 64-byte frames. A frame is bounded by preamble at the front, and an inter-record-gap (IRG) at the end.

The leading slice/flit is organized as two 32-bit (4 byte) sub-fields in the same 64-bit word, these represent the Alice Knowledge, and the Bob Knowledge, in a form that can be tested and swapped. The first slice comprises two 32-bit sub-fields, which may be swapped and updated atomically in a SAW manner.

The slices are all odd/even in the frames. (1/2, 3/4, 5/6, 7/8).

SLICE 1 (index 0) [INITIALIZATION == [000]]

MATCH [PROTOCOL == INITIALIZATION [000]]

RECOGNIZE Initialization Protocol. See State Machine Spec for Petri-Net.

Description We use dual black tokens: one identified with Alice, and one identified with Bob. Both sides of the channel will come up assuming they are Alice, so we need a first level symmetry breaking mechanism.

Short vector We begin by assuming a short name vector (polarity) $N == [00, 01, 10, 11]$. [00] implies uninitialized. We begin by indicating [00] is our name. The other party responds by incrementing this value to [01], and responding with that value to the originator, who takes on the value [10] and reflects that back to the other node which remains as [01]. Alice and Bob may use this distinguished name until they or the other side requests a reset (Verilock will guide us on what to do).

Arbitration If either side finds that the name vector reaches [11], then it requests a second level arbitration. This requires each SmartNIC to generate unique ID's. 256-bit Identifiers (occupy 32 Bytes of a 64-byte frame). These unique identifiers can then be compared on the next cycle of messages and one of them wins, by calling herself Alice [01] and the other side verifies himself as Bob [10]. (Verilock will guide us on what to do).

Identifier Both sides use the short name as their identifier in the first slice of the packet. This distinction is sufficient for any local (channel-only) operations such as liveness and atomic transfer.

RETURN Exit Initialization sequence and return Liveness Tensor packet.

SLICE 1 (index 0) [Hop 1 (LINK) LIVENESS == [001]]

MATCH [PROTOCOL == LIVENESS TENSOR [001]]

MATCH [STATE MACHINE IDENTIFIER == [00000000] && TRANSITION == [00000000]]

RECOGNIZE Basic Liveness Black Tokens.

SWAP (Most significant 32-bits, with Least significant 32-bits).

RETURN Forward on same link: swapped slice.

Purpose This is a basic Liveness cycle. Receive SLICE ONE, fire back SLICE ONE, with the top 32-bits swapped with the bottom 32-bits. Because there will be a second slice (SLICE TWO) right behind it, the SERDES must be ready to receive two slices before it can assume it can SAW the link. Two slices (128 bits, 16 bytes) is therefore the minimum SAW unit.

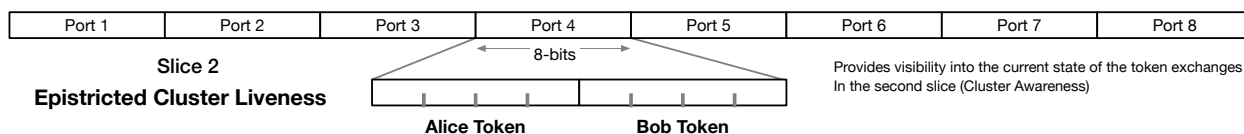
[Future Optimization: Because remaining bytes of SLICE THREE AND FOUR (Index 2..3) are purely 'status' for liveness, and in order to allow the receiving party to pre-empt the stream as quickly as possible, the fire back SLICE ONE includes a 'PRE-EMPTY' capability. This is like a CSMA 'jamming signal' which stops the sender forwarding any more liveness information, and to prepare to receive SLICE ONE (Index 0) of a new frame, for example, to begin a transaction frame as soon as possible.]

CHARLIE: IF Anything Changes before this line, it needs to be reviewed
(Charlie and Paul are unanimous on the above)

SLICE 2 (index 1) [Hop 2 LIVENESS for the Liveness Tensor] 8 bytes of 8 bit Dynamic status (1 byte for each of 8 links).

The link status in slice two (index 1) contains an 8-bit value which comprises two 4-bit Spekkens toy qubits. One of them represents the Petri black token owned by Alice, the other the Petri black token owned by Bob. These are the dual black tokens.

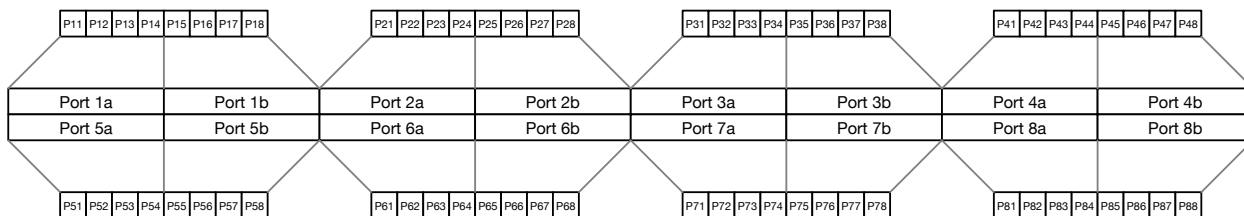
Slice 2 (index 1) contains the same information for all 8 ports connected to this cell. When I get 8 of these (in SLICE 2), I have all the port state of my partners. By sharing this data structure on each link, each of my partners also has the same information I can see on my links.



The interpretation of the Spekkens 4-bits is described in [Dual-SAW-Spekkens.pdf](#) (and references). This document describes the encodings, and rationale for the Stop and Wait (SAW) unit being 1 or 2 slices.

SLICE 3+4 [SUPER CLUSTER LIVENESS 64 links with 2-bit Static status = 128 Bits = 32 Bytes.

[Future Optimization: This slice is optional, and may be pre-empted by the receiver by sending a jam signal in response to the first or second slice.]



(Super Cluster Liveness) 2-bit status assignment to each of 64 'views' of links in the local neighborhood.

See [Liveness-Tensor](#) for details of the 'fish-eye lens' awareness implementation, one hop out (8 links, $3^2 = 9$ cells including self), two hops out (64 links, $5^2 = 25$ cells).

SLICE 5..8 (Index 4..7) [UNIQUE IDENTIFIER] [256-bits/32-Bytes]

Slices 5-8, the last 4 slices of the frame, are optional, and may be pre-empted by the receiver sending a SAW Jam signal in the first, second or 3rd slice.

SLICES 5..8 are part of the Initialization protocol only.

The first hop liveness Tensor contains 8 bits (4 bits for Alice, 4 bits for Bob), Liveness information in a single byte in the first slice.

	Hop #3 Liveness	
	32 bytes (256 bits)	
	8 x 8 = 64	
	8 x 8 x 8 = 512 = 64 Bytes (1 bit per status)	

The second hop liveness Tensor contains $8 \times 8 = 64$ pieces of information about the 2nd hop. Given that this is ‘observing’ each side of the link, this 2 bits per port – Half the number (information content) of the hop-1 information.

The third hop liveness Tensor contains $8 \times 8 \times 8 = 512$ pieces of information about the 3rd hop. This is the minimum number of hops possible for a cell to see a view of a connection back to itself (on another port).

Even with only 1 bit specifying the port information, this is still 512 bits (64 bytes) which doesn’t fit in the 32 bytes available. Alternatives include: (a) extend the minimum packet size to 96 or 128 Bytes. (b) don’t include 3rd-hop visibility (extend the 2-hop visibility instead). (c) a hybrid: with Alice programmed under normal circumstances to ‘accept’ the first hop liveness tensor (2nd slice), and the second hop liveness tensor (3rd and 4th slices), but to ‘reject’ the 5th through 12th slice, so that we minimize the link occupation time with status messages. The liveness tensor is (essentially) status information. There are two possibilities:

1. Bob does the comparison between the old tensor and the new tensor (at any depth) on his side of the link, and indicates with a ‘new’ (or dirty bit) in an early part of the liveness tensor that something has changed, or Bob Just sends it, with the changes in a 64 or 128 packet, ‘imposing’ the information on Alice, whether she is ready to receive it or not.
2. Alice requests the information from Bob, either periodically when she is not busy, or when Bob indicates that something has changed, she ‘accepts’ the changed information in the subsequent slices. This allows Alice to always be an active enabler of what Bob sends her, deciding, for example, to accept the new liveness tensor status, or to say no, and instead begin a new transaction that she has learned about from her side.

Frame Interrupt/Abort

There are two basic types of protocol: (1) Status Exchange Information. E.g. Liveness Tensor shown here, and (2) Execution Operations (described in the State-Machine.pdf document).

Status exchange information is sent as idle information whenever the link is not active. It contains status information, such as the liveness tensor, consensus tensor, and congestion tensor.

These status protocols can be interrupted/ pre-empted, on an even numbered incoming slice (2nd slice, 4th slice, or 6th slice of the protocol), and the other side will terminate the frame and begin a new one.

Forwarding/Cut-Through Routing

Where should we put the keys which trigger the lookups for the routing table? Conventional (A2A) protocols (Ethernet/IP) put destination addresses first so the forwarding engine can get cracking.

The slice engine is capable of handling fewer slices than required by legacy protocols (64 bytes). This lets us handle *local* (low-voltage) operations first and *long distance* (high voltage) operations second. We call these ‘underlay’ or ‘sub-layer 2’ (SL2) protocols (so as not to get confused with IEEE’s layer 1 and 2).

The slice engine is (by definition) for directly connected (not via switches or routers) interconnects, which allows us to make assumptions other NICs cannot, such as not needing to be ‘compatible’ with what switches expect⁴. In conventional networks, forwarding occurs only on minimum size packets (64 bytes). In the FPGA substructure network (FSN), packets do not need to conform to the constraints of legacy switches.

⁴The interoperability specification is part of our security architecture. If we find ourselves connected to conventional Ethernet switches or routers, we simply shut down that port and refuse to talk our ‘low-voltage’ protocol on that port. This satisfies two goals: (1) maintain our confinement guarantees in our network enclave/FPGA substructure, and (2) do no harm if we are accidentally connected to a legacy device. The document (transformer.pdf) specifies the ‘transformer’ which provides an impedance matched (insulated) protocol/energy transfer between the low reachability (low voltage) side of the network, and the high-voltage (high reachability) side of the network - which comprise legacy IP.

FSN packets can be terminated at 1, 2, 4 or 8 slices (8, 16, 32 or 64 Bytes)⁵ The Slice Engine is capable of forwarding frames, but that's only needed for a routing layer, which means ether conventional dst/src addressing, or a key/value lookup. In a name-based routing, everything is a UUID. The forwarding principle for the Slice Engine is:

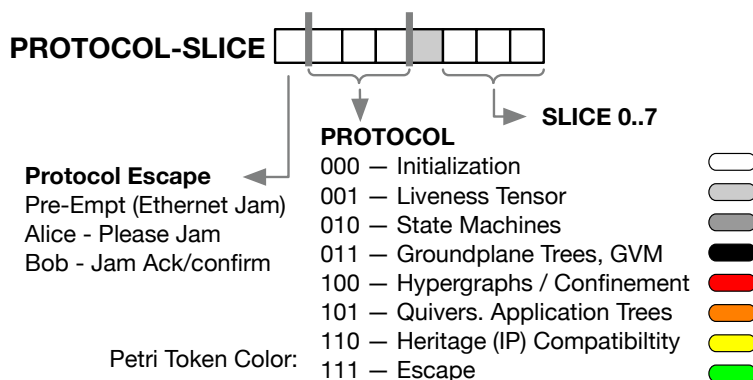
LINK LIVENESS [First slice] is link only—should not be forwarded. Each link is an autonomous agent.

CLUSTER LIVENESS [Second slice] Provides dynamic liveness status for the 8 ports connected directly (1-hop). These ‘interactions’ are shared on each link, there is no point in forwarding the second slice, unless there is a failure, in which case, the topology awareness is useful from perspective from both sides of the link

SUPER-CLUSTER LIVENESS [Third and fourth slices] provide 2-bit (static) liveness status (current ‘knowledge’) for all 8×8 links two hops out. (64 links). This static liveness information can be arranged to be sent *only* when something changes. For example, using NOT-XOR of the Epistricted Spekkens states will give 0 when the link is connected and entangled, but 1 when the link goes offline or the entanglement is lost (is no longer 01 or 10). This SL2 protocol is also not forwarded, because this data structure is effectively ‘sharing’ each of the 8 ports’ information in a single data structure (2-slices, 128 bits (32 bytes)).

UNIQUE IDENTIFIER is Local to the immediate port(s). These are secrets, and should *never* be forwarded. They can, however, be exchanged in the initialization/authentication protocol, so that neighbors 1 hop out can be ‘members’ of your tile. Cells can test to see if their neighbor cells are part of the cluster they are part of, without revealing those unique identifiers, using a Zero Knowledge Proof⁶

HUANG / KLEINBERG FORWARDING (Future enhancement) ‘fast forwarding through cells’. Packets are forwarded directly from an incoming port to an outgoing port, without any processing whatsoever. A ‘logical’ wire. In the spirit of layer 1 switches.



Frame Dynamics – Link Status

The Link is never idle Link Status Information is ‘voluntarily sent’ anytime the link ‘appears free’. I.e. you have a SAW black token in hand. Alice provides Bob with a SAW (a Petri black token). This alone can be used to keep the link alive. However, this is Alice’s black token. She can do whatever she wants with it, including tear it down and stop sending it. There can be a *second* black token – which Bob provides Alice. This is an additional source of ‘events’ to keep the link alive. This is Bob’s black token. He can do whatever he wants with it, including tear it down and stop sending it.

Two Atomically Independent Frames. Each of these frames are captured by the SerDes, modified by the rewriting rule, and sent back. One of these frames is ‘owned’ by Alice, the other frame is ‘owned’ by Bob. Each of them may be pre-empted independently, may progress their state machine, or may reverse their state machine, independently of the other.

⁵Although the mesh network ports on a SmartNIC should never be connected to switches, 1, 2 or 4 slice packets will appear as Runt frames to IEEE 802.3 devices, and (in theory) be incompatible, but not cause problems.

⁶[ZKP](#). allows one party to prove to another party that a given statement is true without revealing additional information.

Frames are sent in 8-byte slices. Two slices (total of 16 bytes) are transmitted at once, before the receiver looks at the pre-empt (signal jam) bit in the top of packet. The frame (slice transmission) can be pre-empted at slice markers 2, 4 or 6. (There could be only 2 choices: 2 or 4), if the jam signal is not asserted, the full 8 slices will be sent. If the frame is pre-empted, then the transmitter stops transmitting the status information in the rest of the frame. And waits for the next frame to be sent by the other side of the link.

Frame Dynamics – State Transfer

The Link is never idle Each side may stop sending link status information, and send atomic transfer information instead. Actual atomic transfer Information takes priority over link status information, and may be sent only using the rules of the State Machine Engine, using Petri black tokens, for some payload.

Sender Pre-emption The sender can choose to truncate the outgoing frame at any time, for example when local application fragments (transactions) have been requested locally. The sender can also monitor the incoming packet from the other side, and if the receiver has sent a pre-emption request, can decide to honor the pre-emption, and wait for the next frame, or can continue to send all or a fraction of the remaining slices in that frame. While the latter is simpler in logic, the former provides potential performance improvements for use cases that can benefit from extreme low latency.

Optimization Although minimum latency will be achieved (for application fragments) by the receiver employing the pre-empt (jamming) signal, there may be times when the sender has noticed (or is notified of a link status change on one of his neighbor connections), and may decide to send the latest information across the link, ignoring the pre-empt request by the receiver.

State Machines cannot be pre-empted. Each state machine (1-State, 2-State, 4-State and 8 State is required to run to completion – either in the forward direction, to complete their roundtrip cycles, or in the reverse direction to ‘uncomplete’ the state machine state changes back to a quiescent state before the next operation (Status or State Transfer) is initiated.

Atomic Dynamics

- Each black token operates completely independently as dual concurrent Petri black tokens. Alice can be swapping and circulating Bob’s black token even though she hasn’t initiated her black token yet, and visa versa.
- Each black token exchange is done ‘atomically’ – Alice’s 32-bits in the first slice are swapped with Bob’s 32 bits in the first slice of the alternating frame.
- Normal operation is when the Alice black token is sent in one Frame, and the Bob black token is sent in a completely separate frame. There are two frames ‘on the wire’ (or more accurately, ‘in the system’) at the same time.
- If for any reason one of the frames is lost or corrupted (or more likely, the software has failed to maintain liveness), then there is still another ‘circulating’ black token that can be used to evaluate link liveness, and continue to be circulated. Only Alice can cease circulating her black token (under ordinary operating conditions), and only Bob can cease sending his black token.
- Each frame is a ‘separate’ atomic unit. Maximum throughput (and minimum latency) can be achieved by having two atomically independent frames ‘occupy’ the bidirectional link

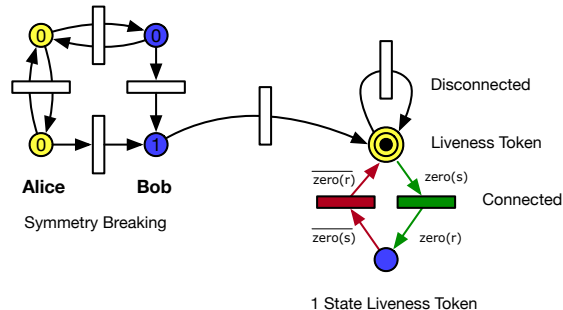
The ontic state of the link is carried in the packet, and a copy (the epistricted state) is deposited in memory in both Alice and Bob independently. There are three Petri places - (1) the input (last bits received off the wire), (2) The sampling state, and (3) the output (last bits sent on the wire). Corresponding to the figure shown in [Dual-SAW-Spekkens.pdf](#)

1-State Transfer State Machine [001] Liveness

The simplest and most common state machine – ‘LIVENESS’. Opening TRANSITION $zero$ [001] Closing TRANSITION $\overline{zero}$. [101]. Yellow circles identify Alice places; Blue circles identify Bob places. When there is no activity on the link, we continuously maintain liveness with this mechanism.

Liveness Protocol – Timeless

1. $zero$. Opening symmetric liveness query.
2. $\overline{zero}$. Closing symmetric liveness query.

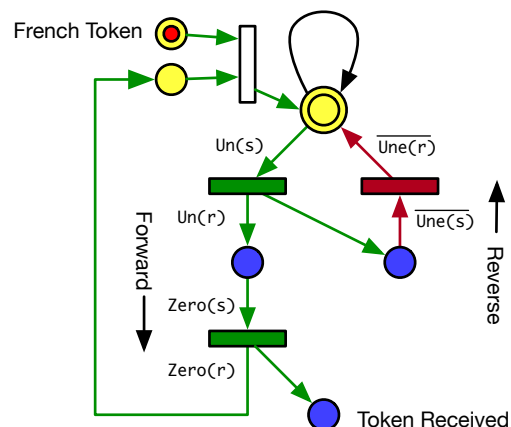


1-State Petri net Model (Liveness)

2-State Transfer State Machine [010]

The 2-State transfer (2ST) protocol is a simple send / acknowledge. It transfers a single black token (colored here in red/French to visually differentiate it) to the other side of the link. The black token could be a simple data payload. Not a transactional payload. The Initialization/Symmetry breaking Petri net (included in the first example above) is assumed in this and all Petri Net's below. Reverse protocol is trivial – simple rejection and ‘undo’ of the operation.

1. un Alice speaks to Bob, confirms symmetry is broken. Understood consistently on both sides of the link.
2. $\overline{un}$, when Bob Speaks to Alice. This enables error-checking for the symmetry-breaking protocol.



2-State Transfer Petri net Model

4-State Transfer State Machine [011]

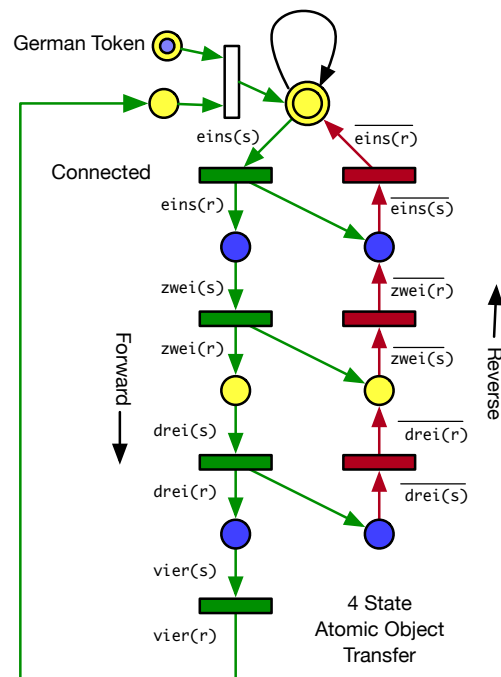
The 4-State Transfer (4ST) protocol requires 4 messages to complete. The protocol uses german number labels to indicate that a blue (german) but actually black token is being transferred across the link.

Forward DIRECTION [01]

1. $eins$ Alice has an black token she wishes to send to Bob, and puts her black token on the wire.
2. $zwei$ Bob receives and sends; Bob-saw-black-token.
3. $drei$. Bob responds to Alice, now Alice knows that Bob knows, that the black token is there.
4. $vier$ Transaction complete. Alice now knows that Bob knows, that Alice knows he has the black token.

Reverse DIRECTION [10]

1. $\overline{eins}$ when Alice speaks to Bob, $\overline{un}$, when Bob Speaks to Alice. This enables error-checking for the symmetry-breaking protocol.
2. $\overline{zwei}$ Undoing operation $zwei$
3. $\overline{drei}$ Undoing operation $drei$



4-State Petri net Model (Liveness)

4. $\overline{vier}$ This operation is not needed. If the protocol reaches the *vier* state, then the final operation is *eins*, which completes the transaction and flushes the black token.

4-State Shared State Machine. PROTOCOL [010] IDENTIFIER [100]

The Shared State Machine (Fig. 2) has 8 primary states and 16 transitions. Each transition has three sub-states (initial conditions, action, and final conditions), represented by Snd, $\rightarrow$, and Rcv respectively. The green channel drives the state machine clockwise, and the red channel drives the state machine anti-clockwise. Closing the protocol in pairs guarantees that information is not lost. See [Shared-State-Machine-Specification](#).

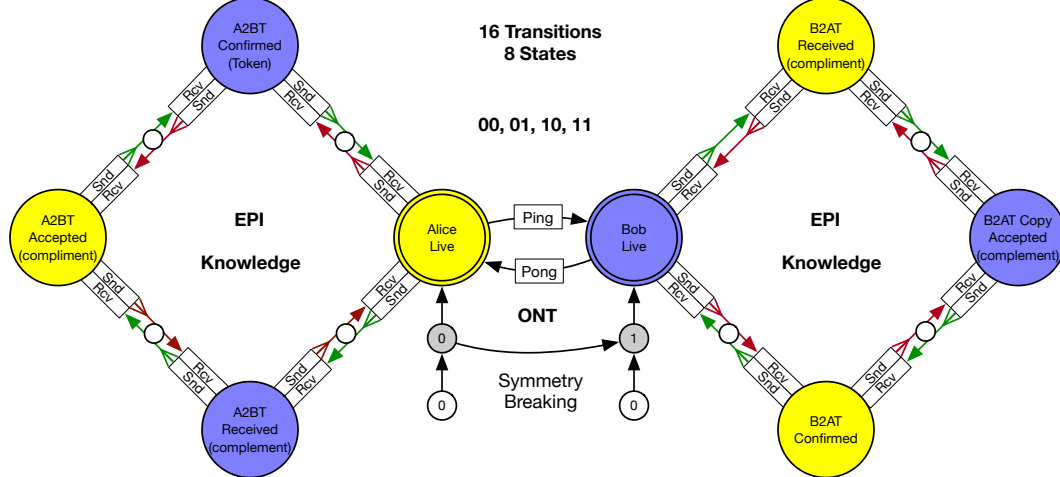


Figure 2: Reversible 4-State Machine Model with Ping Pong

8-State Shared State Machine. PROTOCOL [010] IDENTIFIER [101]

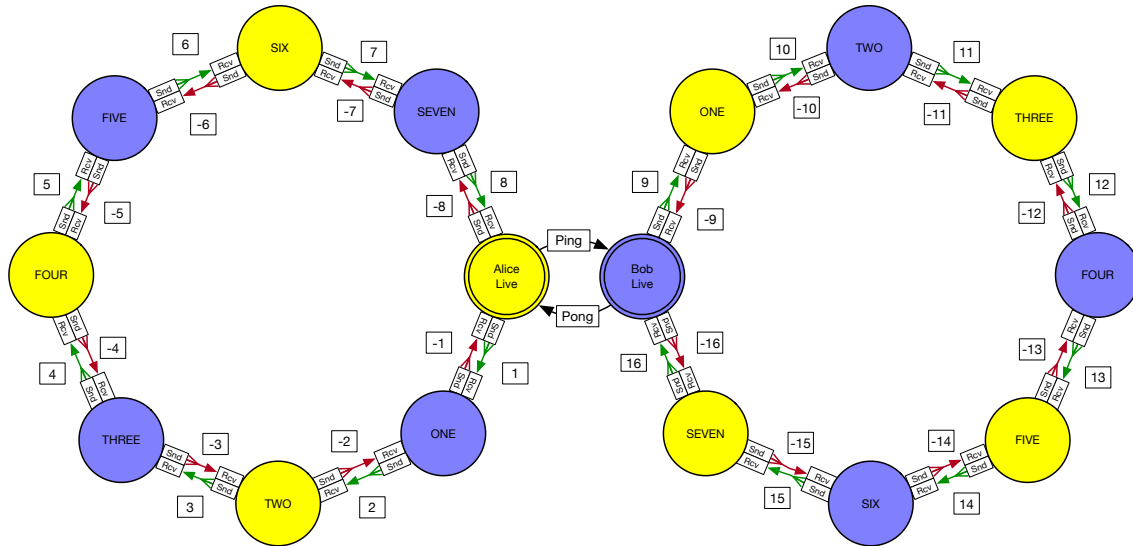


Figure 3: 8 State Shared State Machine

There is only one path back through the state machine if the black token transfer is rejected. This is not a factorial (exponential) expansion of state space, it is linear (proportional to the number of states in the forward path)⁷. At most, it doubles the complexity of the basic mechanism. This operation is not reversible.

This protocol (and one of several other protocols characterized by the owner of the black token) may be used for purposes chosen by the transaction system. For example, in one language (say german), the protocol 'lends' the black token to the other side of the link, while maintaining knowledge on this side of the link that we still 'own' the black token, and expect to get it back when the other side of the link has finished using it.

⁷We go from $2^{16} = 65,535$ possible states for both black tokens driving complex behavior, down to 6 potential Spekkens States and a direction bit to control the whole state machine. The majority of the Spekkens states (an certainly the two EPI views from either side of the link), can be used to identify failures – incorrect steps in the state machine – both forward and backward in time

8-State State Machine [100]

Forward DIRECTION [01]

1. *eins* Begin 1PC.
2. *zwei* Completion of 1PC.
3. *drei* Begin 2PC
4. *vier* Completion of 2PC
5. *funf* Begin 3PC – black token lending (borrowing) initiated across link.
6. *sechs* Completion of 3PC, lending (borrowing) completed across link.
7. *sieben* Begin 4PC – black token ownership transfer (initiated across link).
8. *acht* Completion of 4PC. Black token successfully transferred across link.

Reverse DIRECTION [10]

1. $\overline{eins}$ when Alice speaks to Bob, $\overline{une}$, when Bob Speaks to Alice
2. $\overline{zwei}$ Each of these steps is simply a place to ‘undo’ the operations, and ensure that there is no residual information that can pollute the system.
3. $\overline{drei}$
4. $\overline{vier}$
5. $\overline{funf}$
6. $\overline{sechs}$
7. $\overline{sieben}$
8. $\overline{acht}$

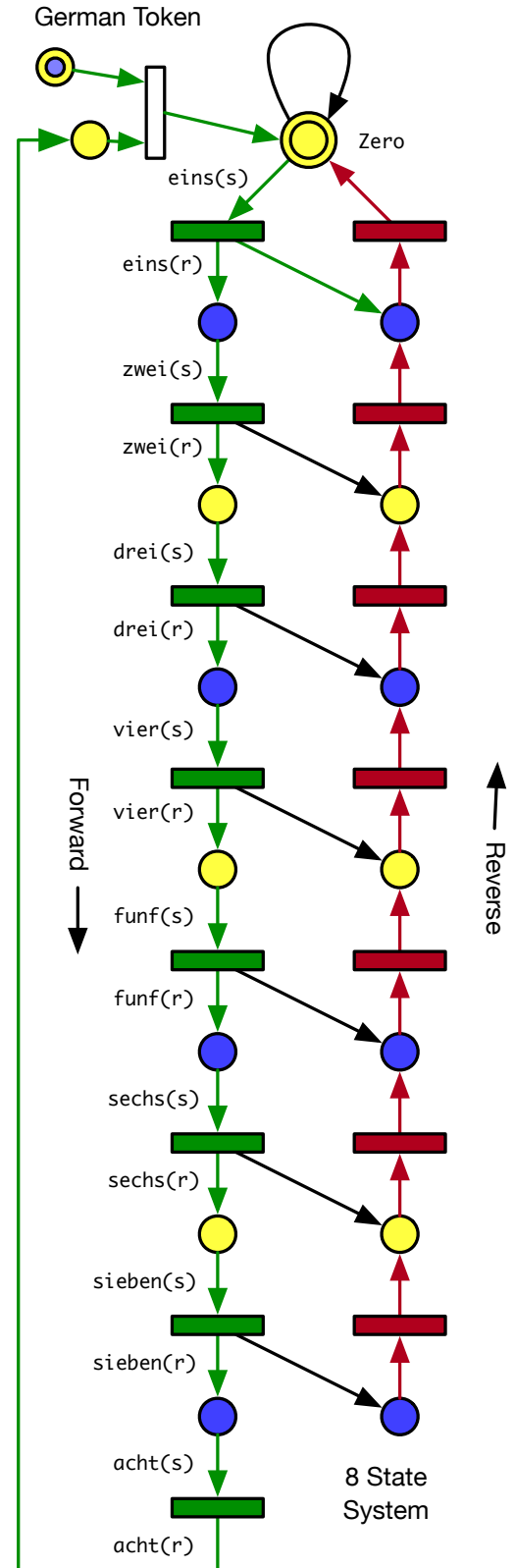
Uninitialized DIRECTION [00]

This condition indicates an error. The DIRECTION State was not initialized before being used. This should never occur.

Wait State – Stuttering DIRECTION [11]

This condition indicates the a single wait state. Typically the protocol will use this as a single step wait state before going from forward to reverse, or from reverse to forward.

Stuttering is an artificial failure mode. It should occur for only one cycle before being automatically shifted back to forward or reverse.



Spekkens Petri Description

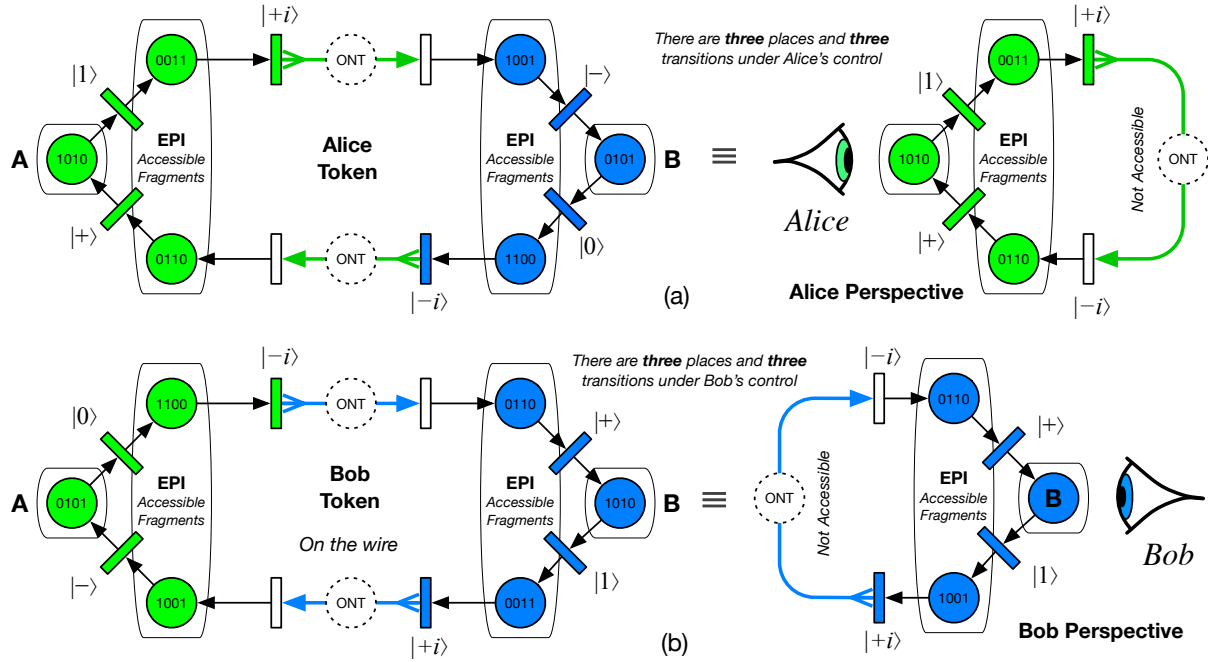


Figure 4: Single Channel. Dual SAW Petri-Spekkens (DSPS) Protocol

A single circulating Petri black token with an *even* number of Petri places and actions will create a stable sequence of actions with the same unchanging state at each transition and place in this Petri Net. Using *two* black tokens, one *owned* by Alice and one *owned* by Bob (Fig. 4) will exhibit alternating ‘dynamic’ liveness at places **A** and **B**.

Using colored tokens, *serializability* for distributed algorithms is resolved for Alice using her token, for Bob using his token, and either waiting for (two phase locking) both tokens. Applications specify ordering constraints, and the interface can ‘hold’ tokens to reliably compose *linearizable* ‘application primitives’.

Spekkens Bits Means ‘Query’

The green squares in the figure below show what is going on in the Spekkens Model.

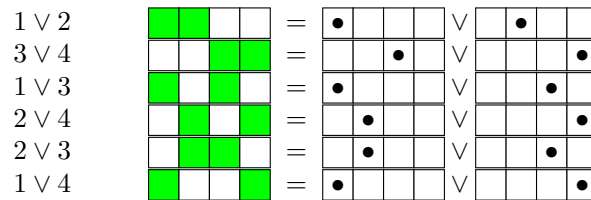


Figure 5: Only allowed measurements are *yes/no* queries on whether *one* of the $a \vee b$ statements is true or not.

Logical queries are always of the form: which of these two statements is true or not? The Knowledge Balance Principle (KBP) guarantees *conserved quantities*⁸. To answer that question, the protocol sends a query to the other side asking one binary question about two potential states.

Spekkens Petri Tokens

Figures on the next page describe the Alice and Bob token bits in Fig 5 above. Interpretation: of the 4 Spekkens bits, 2 bits on the left represent Alice, the 2 bits on the right represent Bob. Bit = (0), means certainty of the ontic state. Bit is one (1), means uncertainty of the ontic state. Because there are 2 bits: 01 and 10 mean 50% uncertainty, while 11 means 100% uncertain, and 00 means completely certain.

⁸“The amount of knowledge one possesses about the ontic state of the system equals the amount of knowledge one lacks”

ALICE TOKEN — Spekkens Interpretation (0 bits = certain, 1-bit = 50%, 11 = 100%)

	Alice Side	Alice Interpretation	Bob Side	Bob Interpretation
Input State to Alice		Input. Alice is 50% uncertain, Bob is -50% uncertain. This is an entangled state.		
Sampling State in Alice		Local EPI state available for sampling (looks like a random time sample).		
Transmitted State in Alice		Local EPI State, ready for sending. Alice information is 100% certain, Bob information is 100% uncertain.		
Compete Temporal Uncertainty (Cannot be measured)		Ontic State On the Wire		
Input State to Bob				Input. Token knowledge about Alice is -50% uncertain, Token knowledge about Bob is 50% uncertain.
Sampling State in Bob				Token knowledge about Both Alice and Bob are 50% uncertain. (Random)
Transmitted State in Alice				Token knowledge about Alice is 100% uncertain. Token knowledge about Bob is 100% Certain.
Compete Temporal Uncertainty (Cannot be measured)				Ontic State On the Wire

BOB TOKEN — Spekkens Interpretation (0 bits = certain, 1-bit = 50%, 11 = 100%)

	Alice Side	Alice Interpretation	Bob Side	Bob Interpretation
Input State to Bob				To. Token Knowledge is is Alice is -50% uncertain, Bob is +50% uncertain. This is an entangled state.
Sampling State in Bob				Local Token EPI state available for sampling (looks like a random time sample). 50% uncertainty for both.
Transmitted State in Bob				Local EPI State, ready for sending. Alice information is 100% uncertain, Bob information is 100% certain.
Compete Temporal Uncertainty (Cannot be measured)				Ontic State On the Wire
Input State to Bob		Input. Token knowledge about Alice is 50% uncertain, Token knowledge about Bob is -50% uncertain.		
Sampling State in Bob		Token knowledge about Both Alice and Bob are 50% uncertain. (Random Probability if you measure now)		
Transmitted State in Bob		Token knowledge about Alice is 100% certain. Token knowledge about Bob is 100% uncertain.		
Compete Temporal Uncertainty (Cannot be measured)		Ontic State On the Wire		

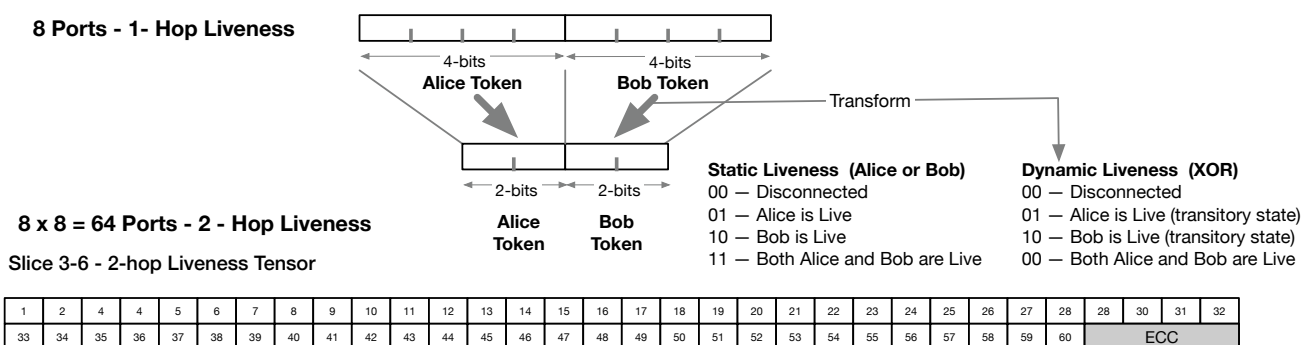
Slice Engine Data Structure for 2nd Hop of Liveness Tensor

[REQUEST FROM CHARLIE: Also, remember that I need an atomic transfer + liveness, two token reversible petri net with all the places and numbers encoded. I don't see that in the file.]

There are four bits in an Alice Token (two bits from Alice, and two bits from Bob) which create the entangled Spekkens state for the Alice token. There are four bits in a Bob Token (two bits from Alice, and two bits from Bob) which create the entangled Spekkens state for the Bob token.

If you take the middle two bits from the Alice token, and XOR them with the Middle two bits from the Bob Token, you will get zeros (result of the exclusive-OR operation). This can be used to generate a 'reduced' link state of only two bits, for links that are on the 2nd hop. Shown as 'dynamic liveness' in the figure below.

Zeros mean 'no-change'. Any bit set means something changed. The XOR Maps a lot of things to zero if they don't change.



2-Bit Link State. There are 16 bits (for 8 ports two hops away) one of which doesn't need to be transmitted – because this information was already in an earlier slice. We use this for something like ECC instead – so that we can do early error detection in the case of packet corruption (e.g. amplitude or phase error in the Ethernet Signal.)

There are many more sophisticated data structures that can be designed to fit within the Slice and Frame Structure of the protocol. However, our preference is to stick with the simple, parsimonious and understandable structures:

First Slice contains the Link Liveness.

Second Slice contains the 1-hop cluster liveness (for 8 ports).

Slices 3,4,5,6 contains the 2-hop Tile liveness (for 64 ports).

Two Concurrent Petri Threads in each Frame going through the FPGA

I imagine that you can have two (or more independent threads in the FPGA logic, simply by having the logic fan out from the two tokens in the diagram shown above (one for Alice, one for Bob). The protocol is designed to have two independent (logical) threads anchored on the two separate (independent) pieces of the slice described above.

We therefore need only one set of Petri State Transfer diagrams, because all the combinations of Alice State Transfer Machines driving the Alice Tokens, and the Bob State Transfer Machines, can be *composed* within the FPGA by the logic, with both threads executing on each clock cycle of the FPGA.

Keep in mind that there are really 4 threads of execution control: One governed by Alice (as primary) and Bob (as secondary) in the first slice of the Frame owned by Alice. And One governed by Bob (as primary) and Alice (as secondary) in the first slice of the Frame owned by Bob.

Units of multiprogramming. In FPGA's if you are not going to a FIFO and another execution (the sledgehammer we are trying to avoid). Trying to do everything in the same clock, all we have to do to deal with multiple tokens is to have more places for more tokens.

New Perspectives

There are several aspects of this design which may not initially appear obvious or understandable. This section covers those ‘new intellectual vantage points’⁹.

Resilience

Resilience depends on a particular kind of information: one that is capable of keeping itself instantiated in physical systems. DNA can replicate itself and keep itself in existence for billions of years, is far more resilient than a stone, which can be ground down by weather and entropy, and turn into dust, over a few hundred or thousand years.

Information

Information is the answer to a yes/no question. New knowledge is created by asking new questions. The most basic question in the universe, from one entity to another is ‘are you there?’. This is what our liveness protocol does. An affirmative answer is fleeting, ephemeral. In order to know if this information is still true, we have to keep on asking it. A basic Stop and Wait (SAW) protocol is reciprocal. Alice’s answer ‘yes’ is the same as a new question to Bob. Bob’s answer ‘yes’ is the same as a new answer to Alice. This creates an *Alternating Causality* which can be used as a resource in distributed systems. The first place that we use it is in the topology awareness of our systems.

Knowledge

“Knowledge is information that is capable of perpetuating its own existence”¹⁰. “Knowledge is the most resilient thing in the universe”. Replication creates persistence. Persistence is a basic ground tool for building robust systems.

Knowledge is captured and persisted information. In memory, in storage, replicated on nearby (one hop) or distant (many hop) systems.

Recipes

“Recipes are made of a special kind of information – knowledge – which has the ability to keep its self in existence”. Our recipes are specified by Petri-Nets (some of which are described in this document). Recipes are always ‘paired’ requests and acknowledgements (therefore involving 2 messages). There can be 1 round trip (RT), 2 RT, 3RT or 4RT recipes.¹¹

Error Detection and Correction

A new perspective on early detection and correction. Instead of forward error correction, which work for correlations at distance, we use alternating causality recipes (ACR).

CRC’s and Forward Error Correction (FEC) provide error detection and correction. Depending on the scheme, one, two, or more bits can be detected as having failed (bit flip or phase flip in the protocol). When one-way trip of the signal is long, such as with radio signals to the Voyager spacecraft, the optimum choice is to add substantial ‘redundancy’ to increase the probability that messages will be received correctly. When the path is short, such as between computers within a rack, adding redundancy in this way only increases the length, the wire occupation time, and the latency of the message. Various generations of Ethernet have struggled with this problem. FEC can add 100-200ns to the system data path¹².

Our protocol has a built in engine to execute the recipes). Instead of an open-ended ‘fire and forget’ we instead require every protocol unit to be not only acknowledged, but flow controlled at the slice level (Frame

⁹These have come from many others, but especially Charlie Johnson, Steven Ganz, and Steve Chalmers.

¹⁰See the popular book by Chiara Marletto [The Science of Can and Can’t](#) for this precise distinction between information and knowledge, and her key paper with David Deutsch on [Constructor Theory of Information](#).

¹¹Follows the perspective on concurrent games in Glynn Winskel’s [Making Concurrency Functional](#). [Topos Institute Video](#).

¹²See: [100+ Gb/s Ethernet Forward Error Correction \(FEC\) Analysis](#).

level in the initial prototype). This provides a different perspective: each round trip can compare what it sent last to what the receiver thinks I sent. This closes the loop in ways that are not possible on long links. By having immediate feedback in the link, we can exploit the expected/unexpected nature of the Spekkens toy qubit (6 valid states down from 16), and, verify consistency (from one slice/frame to the next) with the state machine identifier – and most importantly, the transition identifier in conjunction with the direction bit.

Consequently, instead of detect and correct with bits going in the ‘forward in time’ model of FEC, we use at ‘stop and reverse’ model, where state machine metadata are ‘unwound’ back to a quiescent place in the protocol. This is expected to work well for transient errors (most errors in fiber are random, and in copper, most errors are bursty). For more persistent failures (especially for loss of the link – which may masquerade as packet loss), we fail the link and ‘heal around’ via another link. ‘Topology healing’ with 8 links available will allow partitions to be detected and healed far faster than most protocols (such as IP) can detect, creating an effectively ‘unbreakable’ communications substrate.

Atomic Actions

A core concept in the protocol design is ‘ownership’. Ownership belongs to the process that created the execution entity. Alice ‘owns’ all the tokens she creates and can use them to control the Atomic Actions (*serialization*) of her execution, for example, stopping her own execution while she asks Bob to make forward progress on her behalf (similar to RPC). Alice cannot abandon her children. Bob cannot abandon his children. Neither side can ‘keep’ the token for longer than is allowed by the protocol (1,2,4 or 8 roundtrips), otherwise this would be a protocol violation reported to the layers above.

Token Conservation

Tokens are normally not conserved in Petri Nets. Similar to packets not being conserved in conventional networks (which drop, reorder, duplicate or delay them). To perform exactly once-semantics, we need a principle of CQ/EQ (Conserved Quantities/Exchanged Quantities). One way to do that, is by having the link protocol perform *move not copy* in a foolproof/guaranteed way. It would be natural to extend the mechanisms with ‘color’ that control the Petri Tokens so that they are conserved.

Entanglement is a resource. Used correctly, Tokens can be used to count and manage that resource. We propose to do that using colored (typed) tokens. That way, most tokens are treated the same way that conventional (black/indistinguishable) tokens are treated now.

We can guarantee a simple CQ/EQ sufficient for exactly once semantics with a simple token conservation managed on a process by process basis within each cell. More sophisticated CQ/EQ mechanisms will require a method of unique identifier generation be associated (as a conserved payload).

Frame Ownership and Borrowing

Protection from the protocol stalling is improved if each side is responsible for setting up and tearing down their own frames. This means:

There can be none, one or two frames No frames means that link is idle and needs to be restarted. The link is initialized independently by both sides when it first comes up from power-on or after a reset.

One Token - Alice A single Alice token has an EPI view of the link from both sides. Alice’s view is in her output Petri place, Bob’s View is provided to Alice in the first slice of her frame.

One Token - Bob A single Bob token has an EPI view of the link from both sides. Bob’s view is in his output Petri place, Alice View is provided to Bob in the first slice of his frame.

Two Tokens - One owned by Alice, one Owned by Bob . Views of both tokens are carried in the first slice of the frame owned by Alice, AND Views of both tokens are carried in the first slice of the frame owned by Bob. This provides maximal *knowledge*, from two different perspectives.

Either Alice’s or Bob’s Cell agent can fail independently i.e., stop sending frames, or rewriting the token, but the link will remain active with the other frame.

Questions from Steven Ganz

(Doc: [Exactly-Once-Tiles](#), Jun 5, 2022, Steven Ganz).

[Q1] Should the receiver absorb the token before the sender is aware that it has been received? and thus while the transfer is still open from the sender's perspective (but where the sender will eventually be informed that the transfer succeeded)? If so, a 1RT protocol should suffice. If not, we should look to a 2RT protocol. Since we are selling low-latency, we should not thoughtlessly make assumptions that increase latency.

[A1] The simple answer is no. You will not get exactly once semantics 'between layers' if you conflate them in this way. It is better to handle reversibility completely independently, with each layer having conserved quantities within it, and atomic tokens moving between layers in a strict way, just as we do over a link. We deal with this issue in a separate (TBD) document ([layers.pdf](#)).

[Q2] Is tree-based routing still necessary to complete failed neighbor-to-neighbor frame transfers? NOT the same question as whether tree-based routing will still be present for sending messages¹³ across the network.

[A2] No. Not within a tile. – although higher layer tree may be in operation to do routing, this new 'substrate' lives under the routing layer, to providing topology awareness, and link liveness information.

[A2] Yes. Between Tiles. As you have noticed elsewhere, The Captain of each tile is the center cell, it labels itself and its 1-hop neighbors with its own identity. Each cell is therefore (a) the Captain of its' own tile, and (b) the cohort – with 7 other cells, as a 1-hop neighbor to the Captain of its neighbor tile. This system of overlapping tiles provides an economy of mechanism that solves many problems at the same time.

[Q3] Can we put a routing table on the FPGA (we previously assumed that we could)?

[A3] Yes. If it's made simple enough. The previous Rust version of the code contained far too much complexity in the message handling. Most of this complexity can be eliminated in a new design using the Liveness Tensor, both underneath (to provide topology awareness for initial tree building and configuration), and above (to provide healing around failed links and nodes).

[Q4] How are these relative advantages best balanced?

[A4] Understanding the new design requires a change in perspective. We have built an entirely 'new world' underneath what would normally be called 'the routing layer'. This is to exploit the 'low-voltage/low-addressage' perspective in our new business plan for a Layer 2 protocol that is 'Ethernet compatible' without being required to use the 'high-voltage' protocols of destination and source addresses (at Layer 1.5), while providing a *Transformer* perspective (i.e. non-galvanic signal translation to layer 3 – IP protocols).

[A4] The purpose of layer 1.5 is to provide an explicitly limited form of reachability. This not only helps the system be secure (better confinement), it also makes the system more scalable (information for routing is not propagated any further than necessary), and more resilient. We don't need the routing layer to be operational in order to be 'aware' of topology changes (addition and deletion of links), or the addition of new cell resources.

[A4] Limited reachability is one hop. You don't need a routing scheme to go one hop. You don't need a destination address to go one hop (the address is the port you go out on). You can, however, have awareness beyond one hop using Charlie's Liveness Tensor. Charlie has complete knowledge (from the perspective of both sides of the link between Alice and Bob).

[Q5] Is Figure 3 (a) indeed the only workable tile overlap scheme to support exactly-once transfer of frames(#5)? Figure 3 (a) is also more natural in that every cell is the center, and thus controller, of some tile.

¹³Please do not use the word message, it is far too ambiguous. The stringent Atomicity requirements of the entangled state model do not play well with anything that is 'variable length'. A frame is 64 bytes, and a slice is 8 bytes.

[A5] Yes, and Yes. the (counterfactual) constraint is that it must be 1 hop away. That's what defines a tile. With such a scheme, we can provide mechanisms which inhibit a man in the middle attack, and provide 'temporal intimacy', which only works across a single link (although a higher-level protocol can be used across multiple links to make 'temporal awareness' of the topology transitive to a small number of hops. This is a deliberate constraint, for 'low-addressage' systems within racks.

[Q6] How is a tile/controller selected for the transfer, or is the transfer somehow considered to be within both tiles(#6)?

If it is only within one, in spite of the claim that "Charlie (Captain, Coordinator) sees and can coordinate the tokens of all the Alice's and Bob's in his cluster/tile," the captain at the center does not have visibility over everything that happens in the tile, because cells in the tile will be involved in transfers with cells, inside or outside the tile, but considered to be in another tile for purposes of the transfer. How are transfers through the center cell of the tile coordinated(#7)?

[A6] The Coordinator in the center has the responsibility of coordinating all the activity in the tile. Yes there are links (and transfers through them) which the A's and B's can communicate with each other, but you already know they are not C's for this tile, you are. The only valid transfers (of tokens at this sub-level) is between C and one of his directly connected A's or B's. This way, C always 'sees' the important transactions. See details in [Dual-SAW-Spekkens.pdf](#).

[Q7] How do we handle interruptions in the auxiliary communication between Alice or Bob, and Charlie, that take place in fixing the transfer between Alice and Bob (it was possible to fix such a sequence of link failures that took place during tree-based routing)(#8)?

[A7] The primary mechanism is stop and reverse the protocol, to go back to a known good (quiescent) state.

[Q8] What are the advantages and disadvantages of having the resolver selected as the captain of a tile versus selecting a resolver at the beginning of each transfer or only when the transfer is interrupted(#9)?

Sending knowledge balance information should allow for any cell to be a resolver, which seems more robust, but this makes that cell less of a controller (to avoid having too many cooks).

[A8] Do not use knowledge balance information the way that Dale did it. Stick to the Spekkens recipe, as described in [Dual-SAW-Spekkens.pdf](#) Dual SAW Petri Spekkens.

[A8] The beauty of the tile based approach, is every cell is (by definition), the coordinator for its own tile operations. i.e., you are Charlie for your tile. Each of your directly connected neighbors is an Alice or Bob, i.e. you are 'between' each of these 'pairs' of Alice's and Bob's.

[Q9] What specifically does the Captain contribute to the transfer(#10)?

A message transfer is initiated from the sender, since it is the application running in the sender's cell that requests the transfer. What are the steps by which Alice initiates a transfer to Bob, and the steps by which it is completed, with or without interruptions(#11)?

[A9] The Captain sits in the middle between each pair of cells one hop away. It is already entangled with each of its neighbors, and can reorder, or otherwise coordinate the protocols shown in the Dual-Petri-Spekkens figures. Unlike Dragonfly based architectures, this tile based architecture has a built-in symmetry-breaking mechanism. Rather than each node sitting in a ring, and fully connected (by $N(N-1)/2$), eight cells connected in each tile are *not* fully connected. There is always a self-cell (Charlie) at the center of every tile. This is an important resilience and complexity reduction aspect of the architecture.

[A9] Many systems resort to using a central controller. Unlike the Dragonfly approach, we have a central controller automatically, within the tile – it's the 'self-cell'. The resiliency characteristics benefit enormously from this simple (recursive) identity is the self-cell approach, without the need for symmetry breaking beyond the establishment of Alice/Bob on each link. Even then, the dual frame approach means that symmetry is restored again and Alice can be the controller for her actions, and Bob can be the controller for his actions.

[Q10] How would this mediated transfer be modeled in a Petri net such as those of the State Machine Specification document(#12)?

[A10] See the end of the [State-Machine-Specification](#) document, which shows how multiple links are coordinated by a policy (effectively, a software synchronizer) which can begin a multi-round protocol – such as 2PC – and – in real time, ensure that the participants of the tile execute the operations, in an order specified by the application recipes.

[Q11] If captain can hold tokens, can this cause a deadlock if the captains of several overlapping tiles hold each other's tokens(#13)?

[A11] Yes. That's an extremely important point, and needs to be designed around carefully.

[A11] There are two aspects to this question, and several design choices:

- Liveness tokens continue while the link is stalled (stuttering). [Detects failure hazards faster]
- There is a limit to the number of rounds in the recipe (1RT, 2RT, 4RT, 8RT), this can be used to prevent deadlock (if the exchanges are complete in a finite sequence, the recipe is aborted, and reversed back to a quiescent state.

[Q12] Is this knowledge balance information part of liveness, or is it sent to Charlie as part of a 1RT or 2RT transfer(#14)?

[A12] Separate the 'resources' (Atomic Protocol, Slice Engine) in this lower pre-configuration and healing layer, from the upper layer where routing occurs. The 1RT, 2RT, etc., recipes that drive the slice engine are 'recourses' available at the link level, and can be *composed* to perform higher layer functions within the tile only.

[A12] Routing is between tiles, not between cells. The routing layer (above) connects tiles together. All the 'stitching together' of common knowledge is done within the tile. Healing of the trees begins with the cells on either side of the link finding a route around a failed link – within the tile. This is a *dense* interconnect. Most of the time, we expect there to be 8 cells within a tile, and 8 active connections to 1 hop neighbors. This is a very different (and much more LOV) perspective than the depth/breadth search on conventional trees. Those algorithms will still be important for connecting tiles across many hops in the network.

[Q13] What are the advantages and disadvantages of having the resolver selected as the captain of a tile versus selecting a resolver at the beginning of each transfer or only when the transfer is interrupted(#9)?

[A13] You always have a pre-selected resolver in the tile design. Resilience issue is what happens if the 'self' dies, when it is the coordinator for one of it's own actions (recipes)? The answer, is that the self cell can 'preselect' the failover cells it wishes to let take over. The secondary is selected on one of its ports (which it might do, for example, because it has a full compliment of topology out to two or more hops). The tertiary the same. The liveness tensor allows the self-cell to have this awareness.

[Q14] What specifically does the captain contribute to the transfer(#10)?

[A14] In addition to the description above, the captain is able to be a traditional coordinator in a database sense. Also, note the distinction between Carol (the observer/adjudicator in the non-interfering sense) and Charlie, who can actively 'reverse' the order in which operations take place when they are not consistent with the application recipe constraints.

A message transfer is initiated from the sender, since it is the application running in the sender's cell that requests the transfer.

[Q15] What are the steps by which Alice initiates a transfer to Bob, and the steps by which it is completed, with or without interruptions(#11)?

[A15] Alice always initiates transfers through Charlie. Charlie is the engine that controls a multi-round request-response system.

[Q16] How would this mediated transfer be modeled in a Petri net such as those of the State Machine Specification document(#12)?

[A16] See the answer to this in the above sections of this document.

[Q17] If captain can hold tokens, can this cause a deadlock if the captains of several overlapping tiles hold each other's tokens(#13)?

[A17] See discussion on deadlock avoidance earlier in this document. In a nutshell: deadlock is prevented by the neighbor issuing a protocol violation if Charlie doesn't behave according to the rules (no more than 1,2,4 or 8 cycles of the roundtrip slice engine).

[Q18] Is this knowledge balance information part of liveness, or is it sent to Charlie as part of a 1RT or 2RT transfer(#14)?

Later, there is discussion of "Progressive Knowledge Balance Awareness." For Charlie to accomplish this stitching (or for the cells to do this on their own) requires knowledge of where each is in their respective roles of sender and receiver in a multi-phase transfer. This presumably is provided as the 4-bit transition identifier. So the state of the link, from the perspective of each party, is emitted on the token (slice 1).

[Q19] Is the same information encoded in the Spekkens 1st-hop liveness information, which includes 4 bits for each one-hop port, and represents the order of the arrival and departure of Alice's token and of Bob's token2(#15)?

[Q20] does every frame between Alice and Bob go through Charlie, although Charlie only needs the knowledge balance information to complete the atomic transfer(#16)?

[A20] Simple answer: Yes. More complex answer: Charlie can initiate a 'tile' wide multi-round computation

[Q21] Why can't the parties just know that Charlie is their "fixer," and send knowledge balance information to him when there is a problem with the transfer(#17)?

[A21] Yes. Except we need to deal with the hazards of where Charlie fails, or where Alice or Bob Fail after Charlie has done his stuff but this has not yet been committed to 'stable storage'¹⁴.

[Q22] How is exactly-once delivery to be implemented in the presence of cell failure(#18)?

[A22] Conserved Atomic tokens (a new name for the atomic tokens at the link level), are the 'resource'

Steven – 5 Transfer Protocols and Petri Nets

[Q23] How can the Petri nets make buffer capacities explicit in a two-token system(#19)?

[A23] Please eradicate the notion of 'buffer capacities' from your vocabulary. There are no buffers, or FIFOs. What can't be done in a fast entangled frame circulation (of two frames – one for Alice and one for Bob) isn't worth doing.

Queues, or any kind of buffering, is antithetical to ultra-low-latency applications. The Stop and Wait (SAW) protocol must be one unit. Ethernet compatibility would suggest 64 bytes. Modern FPGA implementations process one 8-byte slice at a time, so we could have a SAW unit of one slice¹⁵.

¹⁴we are being a little mischievous here, because storage is never stable, in the conventional sense, except on an increasing probability of survival related to redundancy basis - i.e. what REPLICUS Software tried to do

¹⁵There are some constraints in current implementations of the SERDES, which we may have to live with. From a business perspective it would be much better to use what is available off the shelf right now in terms of available technology, than to try

The industry has optimized for bandwidth for decades. But bandwidth is no longer the scarce resource. Now, it's low-latency. For an ultimate low-latency transaction protocol, the engineering tradeoffs are different. Instead of thinking in terms of arbitrary length buffers to keep pipes full, we think instead in terms of *registers* (8 byte – 64bits), which match the slices processed by the FPGAs. Each 'register set' is matched to some number of round-trips (1, 2, 4, 8) to complete the protocol.

It is difficult to over-emphasize this point. Ultra-Low latency requires SAW units that are smaller than the cable length. When cable lengths are short, there is no point stuffing more than one unit down the wire at once, all you will do is over-run the SERDES and it will drop packets. At 100Gbits/s (100bits/ns), cable lengths are on the order of 1.6m. Even at 1Tbit (1000 bits/ns) cable occupation times for 64byte frames are still 16 cm (~ 6 inches), which is easily long enough to connect adjacent cells in a rack.

Even if the SERDES had enough capacity and buffering, there is a limit of around 40 Gb/s because the limitations of the host processor memory hierarchy on the other side of the PCIe bus.

[Q24] How will those capacities be realized(#20)?

An additional comment: the 0RT Petri net takes TICK as forward, and TOCK as a reversal (these arcs are marked with zero and NOT(zero)). I think this was intended to form a natural progression to the later Petri nets, but doesn't accomplish that. Is there any reason to view TOCK as a reversal of TICK(#21)?

[A24] If what you mean by 'capacity' is throughput or bandwidth, then you have not characterized our protocol correctly. Please forget about buffers, FIFO's or anything variable length. We tried that for five years and failed. The real-time constraints of the bits coming off the wire will overwhelm the the FPGA SerDes. The Wire protocol must be designed with fixed-size entities. This is why the previous team failed.

Steven – 6 Liveness, Lending, and One versus Two Tokens.

As liveness becomes more enriched, the question arises of whether all frames contain liveness information, or just specific "liveness" frames. We previously allowed for atomic transfer frames to maintain the more limited degree of liveness, so it would seem that all frames should carry the extra liveness information.

[Q25] Should all frames carry the additional liveness information(#22)?

[A25] All frames carry 'pairing' information in the EQ/CQ nature of the protocol. That is liveness information, but we don't have the vocabulary (yet) to distinguish simple heartbeats, from actual information transfer, which would also constitute 'liveness' as the multi-round application recipes would imply.

[Q26] Is the tile-based strategy indeed independent of the liveness protocol(#23)?

[A26] Yes. Liveness and Tile operations are different, they should be thought of as independent.

[Q27] What would we expect to cause token reversal at other passes(#24)?

[A27] This is obvious: Any kind of failure in the receiver – hardware (corruption, error detection), software (buffer failure, capacity exhaustion. Our failure philosophy is not to drop packets, but to return them to the sender so they can do the algebraic tracking of information and transition structures.

[Q28] Is there any reason to expect that additional reversal arcs could not have been installed under a single-token protocol(#25)?

[A28] Good catch. This is the single-token protocol. Two token protocols would be fully concurrent. We should make that clear independently of the Petri-Net descriptions (which each describe the thread of a single token).

and change things. Having said that, because we need only be compatible with other SmartNICs like ourselves, and not switches or routers are usually optimized for bandwidth, we have fewer constraints to deal with than what might otherwise be imagined.

[Q29] Apart from the number of tokens, the content of liveness bits might facilitate exactly-once delivery in the face of link (depending on the answer to question #14) or cell failure (depending on the answer to question #18).

[A29] This is not the way we think about it. The Dale knowledge balance information could not work (it is not my job to prove it). What does work (and for which proofs exist) are the Spekkens disjoint qubits.

[Q30] Is liveness synchronous, i.e., does a cell need to hear from all its neighbors before updating its map and/or before responding to any(#26)? Hopefully not (to avoid deadlock)!

[Q31] But if not, how do we know A and B are receiving consistent information about C, or do we care(#27)?

[A31] We know the information is consistent because the Spekkens consistency of each link is propagated by C.

[Q32] What is the motivation for maintaining two tokens(#29)?

[A32] This is described in the Dual Petri Spekkens document (somewhat). Perhaps not as clearly as you might need. This is an instinct, after exploring many hundreds of scenarios that don't work: we discovered that one circulating frame is a causality event - is propagated through the hardware, and software (through the DMA mechanisms), but while pairing the events on a single frame works, it creates this 'static' data structure, where every place, and every transition, in the Petri net results in a 'static' (non-observable) stasis in the circle. Here's the insight: this stasis (unchanging cycle of information does not result in 'observable' change. (although it does create a continuing circulating set of events — but they all create the same 'EPI' result).

[A32] With two tokens, (one owned, initiated, and torn down by Alice, and one owned, initiated, and torn down by Bob), each separate 'slice engine thread' will create a different result – primarily because they are asking the Spekkens questions in the opposite order – which is what the first figure in the Dual-Petri-Spekkens document tries to convey.

[Q33] Is this indeed the benefit of the two-token liveness protocol, or if not, what is(#30)?

Dual SAW Spekkens says: “serializability for distributed algorithms is resolved for Alice using her token, for Bob using his token, and either waiting for (two phase locking) both tokens.” Normally, a semaphore-like mechanism provides exclusive access to a shared resource. In this case, the various parties don't directly share any resource (other than receiving buffers, which are handled by SAW).

[Q34] What specifically is one allowed to do only when you have your own token(#31)?

[A34] You can do anything you want with your token. Insert it into the link (to maintain liveness). Swap it to carry a higher level (information) token.

[Q35] Only when you have another cell's token(#32)?

[Q36] Can you keep either token and give up the other(#33)?

What are the constraints on lending tiles(#34)? The Dual SAW Spekkens document indicates that every link has two tokens, one for each participating cell, that Charlie may forward Alice's token to Bob, and that with Alice's token, Bob can perform an operation requested, presumably by Alice; Bob's token may remain circulating during this time. While Bob has Alice's token, can Charlie still send messages to Alice, but Alice not send messages to Charlie or anybody else until her token is returned(#35)?

[Q37] What if some messages should await hearing back from Bob, but others are unrelated(#36)?

[A37] At the configuration layer, and in the routing layer, we don't use messages. They only elements of the protocol are fixed size objects (64 byte alternating causality frames owed by one side or the other, and 8-byte

slices, which are part of a frame).

Is there any reason to assume that the requested operation should necessarily be on the same tile as opposed to needing to go across the network, presumably using routing tables(#37)?

[Q38] Can the token ever be forwarded outside of the tile(#38)?

[A38] Simple answer: No. The pre-configuration layer (liveness) does not depend on routing to work, its the other way round. The liveness tensor provides information prior to

[Q39] Why do they have to agree on any order(#41)?

[A39] Because all distributed algorithms have some element of ordering

[Q40] Is this ordering really relating to arrival and departure of tokens, or does it relate to applications (see question #51 below)(#42)?

“The model generalizes to any number of different color tokens or any number of stages in a chain, tree, or graph.”

[Q41] Does it really generalize, and if so, how(#43)?

[A41] Proof that this generalizes is in the literature. It would take me some time to find it. You have no reason to believe this is not true, this is question is out of scope for this document. This has been an extensive discussion with Alan, and in the FAQ’s. There are proofs (chains can generalize to DAG’s).

Steven – 7 Multiple Levels

The State Machine Specification document mentions a Link State Petri net, but this is not presented.

[Q42] How is Link State Petri net defined, and how does it relate to the transfer and liveness nets(#47)?

[A42] See earlier in this document. Link state is an ‘alternating causality’ protocol. It solves the fundamental problem of arbitration in distributed systems. Alternating causality guarantees that (a) an arbitration is performed, and resolved in an asynchronous circuit, and (b) that both sides get a fair (50% probability) of

[Q43] Is a cluster the same as a tile(#48)?

[A43] Yes. Except we prefer the name tile, because that is a physical unit of nodes connected one hop away in a rack. The word cluster is ambiguous, it can mean the multicast set that remains after fail

[Q44] What sort of operation would be performed at a cluster(#49)?

Declaring a cell dead seems a likely candidate.

[A44] Ring shunning. As Alan has previously described.

[A44] Clusters (tiles) can perform any kind of operation that require multiple-‘games’ (roundtrips)¹⁶.

[A44] A useful analogy to think about how the superposition engine works is to compare it to a Sudoku puzzle. By having the majority of interactions take place in the FPGA substructure, it is obvious that we can execute the distributed decision process much faster. However a much more important aspect is that the substructure can eliminate all unnecessary interactions with the host processor in the superstructure above the PCIe bus. Eliminating these interactions allows us to think more clearly about [backtracking](#) in our game of distributed Sudoku. On the surface, this looks like a conventional depth first search in a graph. But there is something much more interesting going on: by having multiple ports (say a SmartNIC with 8 transceivers) connecting compute tiles in the Sudoku pattern (one center cell, and 8 cells around the outside). These ‘ports’, and the algorithms, can all be active in parallel, just like in Stephen Wolfram’s [Multiway System](#).

¹⁶An example of the kind of ‘tile-games’ that can be played is [Quantum Sudoku Solver, Algorithm](#), and [Quantum Sudoku](#). You don’t need a real quantum computer if you have our Dual-SAW-Epistricted protocol. See: [Solving Sudoku using Grovers Algorithm](#). This is also related to Glynn Winskel’s [Making Concurrency Functional](#). [Topos Institute Video](#).